

ANTIGRAVITY DEVELOPMENT PLAN

Rural / Village Local Jobs SaaS — Recommended Build Order

Backend-first foundation with a small frontend shell, followed by complete feature modules and secure role-based dashboards.

1. Recommended Overall Order

Phase	Build
1	Project architecture
2	Database/domain design
3	Backend foundation + authentication
4	Basic frontend shell
5	Connect frontend backend
6	Worker features
7	Job Provider features
8	Matching + notifications
9	Admin dashboard
10	Security + testing
11	Deployment

2. Why Backend Should Lead

This platform has important server-side business rules. The backend should be the source of truth for permissions, job state, applications, hiring, ratings, notifications and security.

- Worker / Job Provider / Admin role separation
- Authentication and authorization
- Skills, availability and profiles
- Applications and worker selection
- Automatic FILLED status when capacity is reached
- Automatic EXPIRED status after deadline
- Matching and notifications
- Ratings, reports, restrictions and bans
- Chat permissions and job ownership
- Location and privacy controls

The frontend displays backend state; it must not be trusted to enforce rules such as job capacity or expiration.

3. Build a Small Frontend Shell Early

- Login / registration
- Language selection
- Worker dashboard

- Job Provider dashboard
- Admin dashboard
- Protected routes/navigation
- Basic responsive mobile-first layout

This lets you test API integration continuously without trying to build the entire UI before the backend is ready.

4. Recommended Technical Stack

Layer	Recommendation
Frontend	Next.js + React + Tailwind CSS
Backend	Java + Spring Boot
Database	PostgreSQL
Authentication	Mobile OTP / secure authentication
API	REST API
Chat	WebSocket / realtime messaging
Maps	Geolocation/maps provider
Notifications	Push initially; SMS/WhatsApp later
Storage	Object storage for profile/documents

5. Phase 1 — Architecture & Domain Model

Define the boundaries between frontend, backend, database, authentication, background jobs and external services before feature implementation.

- Worker, Job Provider and Admin roles
- API conventions and error format
- Development/staging/production environments
- Database migration strategy
- Security baseline
- Logging and audit foundation

Initial entities: User, WorkerProfile, ProviderProfile, Skill, WorkerSkill, Availability, Job, JobSkill, Application, Assignment, Review, Conversation, Message, Notification, Report, Dispute, Restriction/Ban, AuditLog.

6. Phase 2 — Database First

- User WorkerProfile / ProviderProfile
- Worker Skills
- Worker Availability
- Provider Jobs
- Job Required Skills
- Worker Application Job
- Job Selected Worker Assignment
- Job Conversation / Messages
- Completed Job Reviews
- User Reports / Disputes / Restrictions
- Admin actions AuditLog

7. Phase 3 — Backend Foundation

- Spring Boot project setup
- PostgreSQL connection and migrations
- Entities/repositories/services/controllers
- Server-side validation
- Standardized API errors
- Authentication
- RBAC: WORKER, JOB_PROVIDER, ADMIN
- Secure configuration/secrets
- Logging and audit foundation
- Basic rate limiting/abuse protection

8. Phase 4 — First End-to-End Vertical Slice

Worker registers creates profile Provider creates job Worker sees job Worker applies Provider selects worker backend checks capacity job becomes FILLED when required capacity is reached.

Make this workflow work end-to-end before adding advanced features. It validates the core marketplace architecture.

9. Phase 5 — Worker Module

- Registration/login
- Language selection
- Profile, skills and experience
- Location and availability
- Expected payment
- Job search/filtering
- Job details
- Apply/withdraw application
- My applications
- My active/completed jobs
- Notifications
- Messages
- Ratings/reviews

10. Phase 6 — Job Provider Module

- Registration/login
- Provider profile
- Create/edit/publish job
- Required workers, skills, location, payment, date, duration, deadline
- Search/filter workers
- View worker profile, rating and verification
- Review applications
- Shortlist/select workers

- Automatic FILLED state
- Cancel/complete job
- Messages/contact
- Ratings/reviews

11. Phase 7 — Matching & Notifications

Match using skills, distance, availability, preferred category, payment fit, experience, reputation and urgency.

- Notify only eligible workers
- Worker chooses radius and categories
- Minimum payment preference
- New matching job notification
- Application/selection notification
- Message notification
- Job reminder
- Cancellation notification
- Rating request

Never send normal matching notifications for FILLED, EXPIRED or CANCELLED jobs.

12. Phase 8 — Job Automation

Condition	Backend action
Selected workers >= required workers	FILLED; close applications; stop normal matching notifications
Deadline passed while OPEN	EXPIRED; remove from active listings; stop applications
Provider cancels	CANCELLED; stop applications/normal matching
Job completed	Enable rating/review flow

Use scheduled/background jobs for deadline checks, reminders and notification tasks. Enforce state transitions server-side.

13. Phase 9 — Chat & Contact

- Job-specific conversation
- Worker provider messaging
- Realtime updates via WebSockets where appropriate
- Message notifications
- Report/block controls
- Privacy-aware calling/contact flow

14. Phase 10 — Ratings, Trust & Bans

Both workers and providers can rate each other after completed work. A single bad rating should not automatically ban an account.

Normal Warning Restricted Suspended Banned

- Consider repeated low ratings and credible reports
- Detect suspicious/fake/retaliatory rating patterns
- Serious safety/fraud/abuse issues can trigger immediate suspension for investigation

- Provide appropriate admin review and appeal
- Record enforcement reason, actor and timestamp in audit logs
- Apply the trust framework to both workers and providers

15. Phase 11 — Three Separate Dashboards

Dashboard	Main functions
Worker	Nearby/recommended jobs, applications, active jobs, availability, profile, skills, ratings, messages, notifications
Job Provider	Create jobs, worker search, applications, selection, active/completed jobs, messages, ratings, notifications
Admin	Users, jobs, verification, reports, disputes, rating moderation, warnings/suspensions/bans, categories, translation

16. Phase 12 — Security

Implement security alongside backend development, not as a final add-on.

- Secure authentication and session/token handling
- Strict RBAC on every protected API
- Least-privilege admin access
- Server-side validation
- Rate limiting and abuse protection
- Secure headers and safe error responses
- Encryption in transit and appropriate encryption at rest
- Secrets outside source code
- Private storage for sensitive files
- Privacy controls for phone numbers and location
- Audit logs for bans, verification and job-state changes
- Database backups and tested restore
- Dependency/security updates
- Monitoring and incident-response process

17. Phase 13 — Multilingual Support

- Start with Marathi + Hindi + English
- Use translation keys instead of hard-coded UI text
- Translate navigation, forms, categories, notifications, help and settings
- Allow language changes from settings
- Use localized notification templates
- Keep user-generated job descriptions separate from system translations

18. Phase 14 — Admin & Operations

- User management
- Worker/provider verification
- Job moderation
- Reports and disputes
- Warnings, restrictions, suspensions and bans
- Rating/review moderation
- Skill/category management
- Translation management
- Notification templates
- Audit/security logs
- Analytics

19. Phase 15 — Testing

Area	Examples
Functional	Registration, profiles, jobs, applications, hiring, reviews

Area	Examples
Business rules	Capacity, FILLED, EXPIRED, cancellation, deadlines
Security	RBAC, unauthorized API access, validation, rate limits, session security
Abuse	Spam, fake applications, suspicious ratings, reports
Mobile	Responsive UI, touch targets, slow-network behavior
Localization	Language switching and translated notifications
Performance	Search, matching, notifications, concurrent users
Recovery	Backups, restore, background-job failure handling

20. Phase 16 — Deployment

- Staging and production environments
- HTTPS/domain
- Frontend + Spring Boot deployment
- PostgreSQL
- Object storage
- Notification services
- Monitoring/logs/alerts
- Automated backups
- CI/CD
- Production security review

21. How to Work in Antigravity

Use small, explicit implementation phases. Do not ask Antigravity to generate the whole SaaS in one prompt.

- Start with architecture and database requirements
- Implement one module at a time
- Build/test after every milestone
- Keep business rules documented
- Never bypass backend authorization
- Review migrations before applying
- Review authentication/security code carefully
- Commit working milestones so you can roll back
- Connect UI to stable API contracts

22. Suggested Build Milestones

Milestone	Definition of done
M1 Foundation	Project runs; database connects; roles/auth exist
M2 Worker	Worker registers, profiles, adds skills and availability
M3 Provider	Provider creates and publishes a job
M4 Marketplace	Worker searches/views/applies; provider reviews applications
M5 Hiring	Provider selects worker; capacity automatically closes job
M6 Expiry	Deadline automation changes open jobs to EXPIRED

Milestone	Definition of done
M7 Notifications	Matching workers receive relevant notifications
M8 Communication	Job-specific chat/contact works
M9 Trust	Reviews, reports, restrictions and admin moderation work
M10 Dashboards	Worker, Provider and Admin dashboards are separate and protected
M11 Security/QA	Security, business rules, mobile and localization tests pass
M12 Pilot	Small local launch with real users and marketplace metrics

23. What NOT to Build First

- AI matching
- Marketplace payments
- Every Indian language
- Every possible job category
- Complex worker teams
- Large analytics suite
- Entire application in one Antigravity generation

24. Final Recommended Sequence

Architecture Database Backend/Auth Small frontend shell Worker Provider First end-to-end flow
 Matching Notifications Job automation Chat Ratings/trust Three dashboards Security hardening
 Testing Deployment Local pilot.

The key milestone is a complete real transaction: Provider posts local job matching worker discovers it applies provider selects job closes when capacity is reached work completes both sides rate.

25. First Task to Give Antigravity

Start with project architecture + database/domain model + authentication/roles. Do not implement the whole platform at once. After reviewing the foundation, build the Worker and Provider vertical slice end-to-end.

26. Key Principle

Build the platform as a secure backend-driven marketplace with a mobile-first frontend. The backend owns business truth; the three dashboards consume authorized APIs; background jobs handle deadlines/notifications; trust and security are built into the core architecture.